

Case Study Report: Software Testing & Maintenance

Design of Test Cases and a Maintenance Plan (Sample App: E-Learning Platform)

1. Aim and Objectives

Aim

The aim of this case study is to design a comprehensive suite of software test cases and develop a robust post-deployment maintenance plan for a sample application, an online E-Learning Platform.

Objectives

- Identify critical application functionalities requiring rigorous testing.
- Design standard functional and non-functional test cases.
- Establish a structured software maintenance plan for the application's lifecycle.
- Outline defect tracking processes to handle system bugs effectively.

2. Project Overview

Continuing with the **E-Learning Platform** as the sample application, the system handles sensitive user data, video streaming, and financial transactions. A single bug in the payment gateway or an unexpected downtime could lead to significant financial and reputational loss. Therefore, thorough testing before release and a strategic maintenance plan post-release are paramount for long-term viability.

3. Test Strategy and Approach

The testing approach relies on a combination of manual and automated testing to ensure comprehensive coverage across the application. The primary phases of testing include:

- **Unit Testing:** Verifying individual components (e.g., login function, search algorithm).
- **Integration Testing:** Ensuring the frontend communicates correctly with the backend database and APIs.
- **System Testing:** Testing the fully integrated application in an environment that mirrors production.
- **User Acceptance Testing (UAT):** Validating end-to-end business flows by simulating real user behavior.

4. Design of Test Cases

The following table outlines a sample set of core test cases designed to evaluate both the functionality and security of the platform.

Test Case ID	Module	Test Description / Steps	Expected Result
TC-001	Authentication	Enter valid email and password on the login page and click "Login".	User is successfully authenticated and redirected to the main dashboard.
TC-002	Authentication	Attempt login using an unregistered email or incorrect password.	System denies access and displays an "Invalid credentials" error message.
TC-003	Payment	Initiate a course purchase using a valid mock credit card.	Payment is processed successfully, receipt is generated, and course is unlocked.
TC-004	Video Player	Click on a purchased course video and play it on a mobile network connection.	Video loads within 3 seconds and adjusts resolution based on bandwidth without buffering.
TC-005	Security	Input basic SQL injection scripts (e.g., ' OR 1=1--) into the search bar.	System sanitizes input, rejects the query, and prevents database unauthorized access.

5. Maintenance Plan

Post-deployment, the software enters the maintenance phase. The maintenance plan categorizes activities into four distinct types to ensure continuous system health:

- **Corrective Maintenance:** Fixing reactive bugs and glitches reported by end-users (e.g., resolving a broken video link).
- **Adaptive Maintenance:** Updating the software to adapt to changing environments (e.g., updating the app to support a new iOS or Android version, or upgrading the database server).
- **Perfective Maintenance:** Enhancing existing features based on user feedback (e.g., adding a "Dark Mode" to the user interface).
- **Preventive Maintenance:** Restructuring code (refactoring) and optimizing databases to prevent future performance degradation and security vulnerabilities.

6. Defect Tracking and Management

To implement the maintenance plan efficiently, a strict defect tracking workflow is established. Using an issue tracking tool (like Jira or Bugzilla), bugs will follow a standardized life cycle:

1. **New:** A user or QA tester reports a bug.
2. **Assigned:** The Project Manager assigns the bug to a specific developer.
3. **In Progress:** The developer is actively fixing the issue.
4. **Fixed:** The developer completes the code correction.
5. **Verified:** The QA engineer re-tests the fix to ensure the bug is resolved.
6. **Closed:** The issue is documented and removed from the active queue.

7. Results

The design of these test cases and the establishment of the maintenance plan resulted in a highly reliable framework for quality assurance. By systematically executing the defined test scenarios (TC-001 through TC-005), the team can identify and resolve critical functional and security defects before production release. Furthermore, the 4-tier maintenance plan ensures that post-launch, the application remains adaptable to new technologies, secure against emerging threats, and capable of addressing user-reported defects with minimal downtime.

8. Conclusion

Software development does not end at deployment. A rigorous testing phase is critical to delivering a stable and secure application, while a well-structured maintenance plan ensures its longevity and continuous improvement. For this E-Learning Platform, the detailed test cases safeguard the user experience and financial transactions. Meanwhile, the robust maintenance and defect management strategies guarantee that the software can evolve alongside user needs and technological advancements, ultimately achieving high user satisfaction and a strong return on investment.